



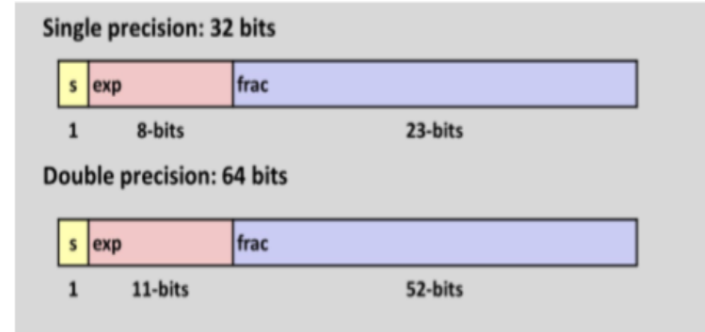
CMPT 295

Unit - Data Representation

Lecture 8 – Representing real numbers in memory cont'd –

IEEE 754 standard for floating point representation,
rounding, their arithmetic operations and `float` in C

Last Lecture



➤ IEEE 754 standard - R2B (X) :

1. Convert the given real number **R** into a binary number (with integral and fractional parts as well as a binary point) using the **positional notation**.

2. Normalize this binary number into the **binary numerical form**

$$V = (-1)^s M 2^E$$

3. Convert this binary number expressed in the **binary numerical form V** into an **IEEE 754 floating point number** (i.e., a bit pattern **B**) using the **appropriate** equations.

➤ **Normalized** => **exp** within range: [00000001 .. 11111110]

➤ **E** = **exp** - **bias** where **bias** = $2^{k-1} - 1$

➤ **M** = 1. + **frac**

➤ IEEE 754 standard - B2R (X) :

1. Reverse the above algorithm.

Today's Menu

- Storing data in memory

- “Under the Hood” - **von Neumann** architecture

- Bits and bytes in memory

Note that we do not know yet what data these bits represent!

- How to diagram memory (used in this course and other references)

- reviewed in Lab 0** ➤ How to express memory content -> **using binary and hexadecimal numeral system**

- Order of bytes in memory -> **endian**

- Bit manipulation – bitwise operations

- Boolean algebra + shifting

We shall start giving meaning to bits -> what data they represent.

- What kind of data do bits (in memory) represent -> **encoding scheme**

- Bits representing (encoding) integral numbers

- reviewed in Lab 1** ➤ Unsigned and signed

- practiced in Lab 2** ➤ Converting, expanding and truncating – Impact of fixed size memory

- Arithmetic operations

- Bits representing (encoding) real numbers

- IEEE floating point representation (cont'd)

- Floating point in C – casting, rounding, addition, ...

Demo

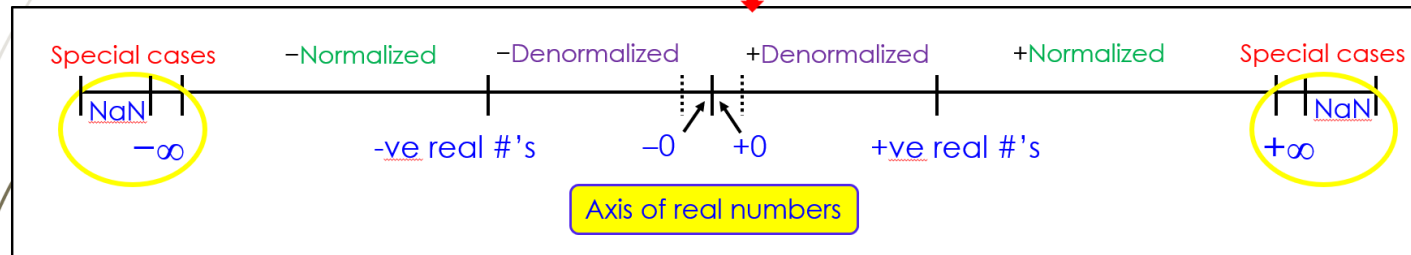
In Lecture 5, we saw that ...

encoding Representing real numbers in memory

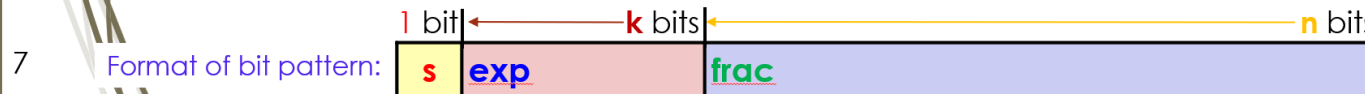
B2R(x) and R2B(x)

- Bit pattern $\Leftrightarrow$ Floating point (IEEE 754) $\Leftrightarrow$ Real numbers
or bit vector
- Encoding scheme called IEEE 754 standard for floating point representation
- Allows us to represent a **large range** of real numbers (but not all):

And most of these
real numbers are
approximated.



- And all represented real numbers (Normalized, Denormalized or Special cases), once converted to a bit pattern, will have this format in memory:



- But, so far, all **R** were represented **exactly**, they were not **approximated**!

Example of **R** encoded approximately, using **IEEE 754** floating point representation:

➡ How would **47.28** be encoded as IEEE 754 floating point number?

R2B (X)

1. Convert 47.28 to binary number using the **positional notation**

➡ $47 = 101111_2$

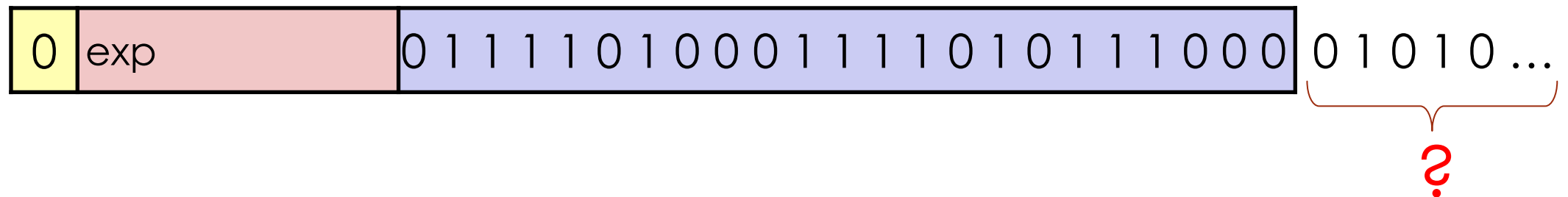
$.28 = .01000111101011100001_2$

➡ $.28 = .01000111101011100001010001111010111000010100011110101110000101..._2$

2. Normalize binary number:

$101111.01000111101011100001010001111010111000010100011110101110000101..._2 \times 2^0$

$\Rightarrow 1.011110100011110101110000101000111101011100001010001111010111000..._2 \times 2^5$



Rounding Rules

This **selection** is done by looking at the bit pattern around the **rounding position**.

The **worth** of a bit is the value of 2^x where x is associated with its position in the bit pattern, even though the bit itself may be 0.

➤ First, identify the bit at **rounding position**

➤ Then select which kind of **rounding** we must perform:

1. **Round up**

10.0**0**110₂

➤ If the value of the bits to the right of the bit at **rounding position** is $>$ half of the **worth** of the bit at **rounding position**, then we **round up** by adding **1** to the bit at **rounding position** and by discarding the bits to the right of the bit at **rounding position**

2. **Round down**

10.0**0**011₂

➤ If the value of the bits to the right of the bit at **rounding position** is $<$ half of the **worth** of the bit at **rounding position**, then we **round down** by discarding the bits to the right of the bit at **rounding position**

3. **Round to even number**

➤ If the value of the bits to the right of the bit at **rounding position** is exactly half of the **worth** of the bit at **rounding position**, i.e., when these bits are 100...0₂, then we **round** such that the bit at the **rounding position** becomes 0

10.1**1**100₂

➤ If the bit at **rounding position** is 1 $\Rightarrow$ then we **round up to even number** by **rounding up** (see **Round up** above)

10.1**0**100₂

➤ If the bit at **rounding position** is already 0 $\Rightarrow$ then we **round down to even number** by **rounding down** (see **Round down** above)

Examples of Rounding Rules

➤ Example: **rounding position** -> round to nearest $1/4$ ($1/2^2 = 2^{-2}$)

worth of this bit is $1/4$ ($1/2^2 = 2^{-2}$)

half of the **worth** is _____ ?

Value	Binary	Action	Rounded	Rounded Value	Error introduced by the rounding process
$2 \frac{3}{32}$	$10.0\mathbf{0}011_2$	< half of the worth ->down	10.00_2	2	-3/32
$2 \frac{3}{16}$	$10.0\mathbf{0}110_2$	> half of the worth ->up	10.01_2	$2 \frac{1}{4} = 2 \frac{4}{16}$	+1/16
$2 \frac{9}{32}$	$10.0\mathbf{1}001_2$	< half of the worth ->down	10.01_2	$2 \frac{1}{4} = 2 \frac{8}{32}$	-1/32
$2 \frac{7}{16}$	$10.0\mathbf{1}110_2$	> half of the worth ->up	10.10_2	$2 \frac{1}{2} = 2 \frac{8}{16}$	+1/16
$2 \frac{7}{8}$	$10.1\mathbf{1}100_2$	= half of the worth ->up to even	11.00_2	3	+1/8
$2 \frac{5}{8}$	$10.1\mathbf{0}100_2$	= half of the worth ->down to even	10.10_2	$2 \frac{1}{2} = 2 \frac{4}{8}$	-1/8

➤ **value of a bit** versus **worth of a bit**

Effect of approximation

Back to IEEE 754 floating point representation

In the process of converting real numbers **R** to IEEE floating point numbers (i.e., bit patterns **B** stored in fixed-size memory), we apply these same rounding rules ...

Using the same numbers we had in our example:

Binary

10.0**0**011₂

10.0**0**110₂

10.0**1**001₂

10.0**1**110₂

10.1**1**100₂

10.1**0**100₂

Imagine this is the 23rd bit of the **frac**
=> rounding position

and this is the 24th bit

Back to our example of **R** encoded approximately,
using IEEE 754 floating point representation:

➡ How would **47.28** be encoded as IEEE 754 floating point number?

R2B (X)

1. Convert 47.28 to binary number using the **positional notation**

➡ $47 = 101111_2$

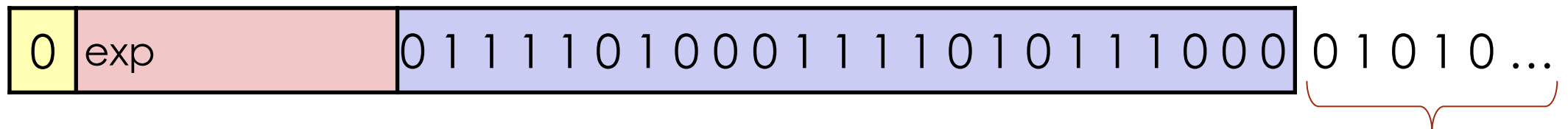
$.28 = .01000111101011100001_2$

➡ $.28 = .01000111101011100001010001111010111000010100011110101110000101..._2$

2. Normalize binary number:

$101111.01000111101011100001010001111010111000010100011110101110000101..._2 \times 2^0$

$\Rightarrow 1.011110100011110101110000101000111101011100001010001111010111000..._2 \times 2^5$



Let's practice converting and rounding!

IEEE floating point representation
 - single precision - $k=8$
 $V = (-1)^s M 2^E$
 $M = 1 + \text{frac}$ (normalized)
 $E = \text{exp} - \text{bias}$
 $\text{bias} = 2^{k-1} - 1 = 2^7 - 1 = 127$

How would **346.62** be encoded as IEEE floating point number (single precision) in memory?

1) R2B (346.62_{10}) using positional notation:

$101011010.1001111010110000101010_2$

2) Normalize the binary representation of 346.62_{10} :

$101011010.1001111010110000101010_2 \times 2^0 \leftarrow E$

$1.010110101001111010110000101010_2 \times 2^8$

implied
 $\therefore$ not part
 of bit
 vector
 (IEEE)

3) $E = 8$

$\text{bias} = 127$

$\text{exp} = E + \text{bias} = 8 + 127 = 135_{10} \rightarrow \text{R2B}(135_{10}) = 1000111_2$

$M = 1 + \text{frac}$

$\text{frac} = 010110101001111010110000101010...$

Rounding down because the bits that are discarded, i.e., $00101010... < 10000000...$
 $< \text{half of } 1/2^{23}$
 $= 1/2^{24}$

$\therefore 346.62_{10} \approx 0 \text{ } 1000111 \text{ } 0101101010011110101100$

$\Rightarrow 0x432d4f5c \rightarrow$

More **rounding** practice!

kind of **rounding**:

1. 0100011001000000111001110000011**0**10000
2. 0100011001000000111001110000011**1**010101
3. 0100011001000000111001110000011**1**11101
4. 1100011111000100110001100010011**0**100001
5. 1100011111000100110001100010011**0**000001
6. 0100000001000000111001110000011**1**10000

Error introduced by the rounding process

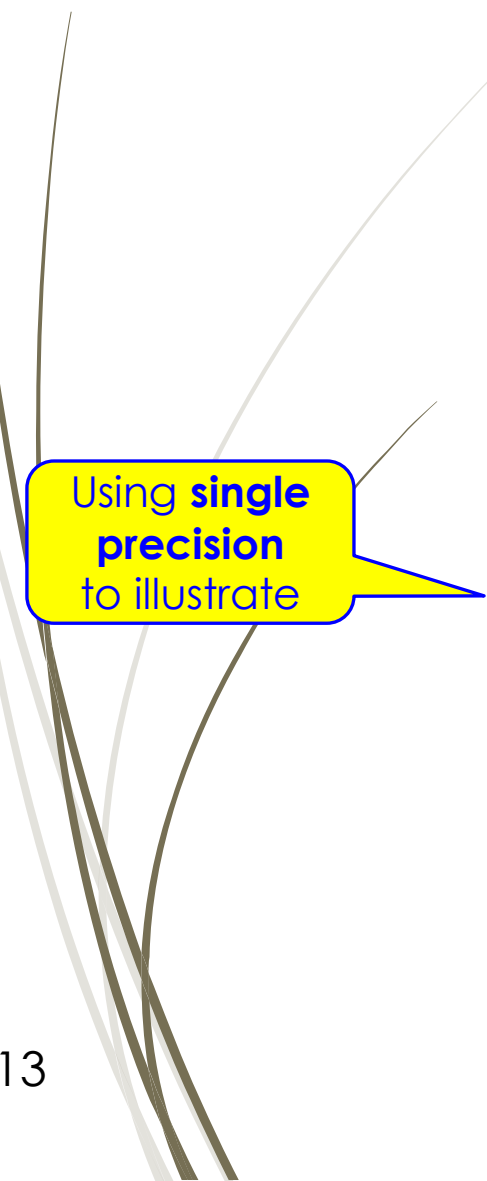
- Let's compute the **minimum value** of the **error** introduced by the **rounding process** since **346.62** can only be **approximated** when encoded as an **IEEE 754 floating point representation**

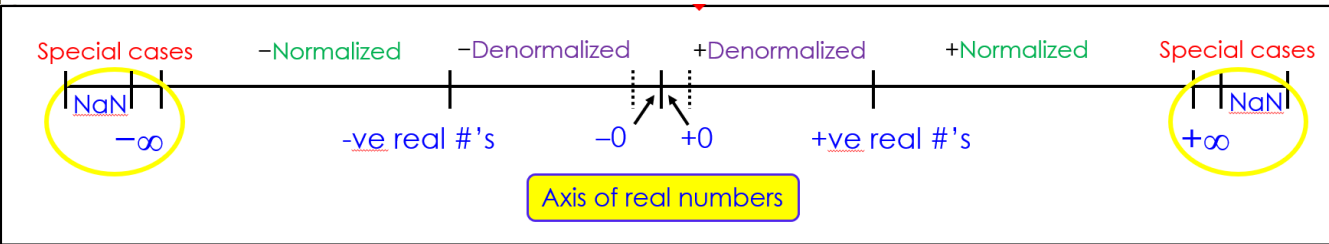
2^{-23}



- frac**: 0101101...0**0**00101010...
- The **error** is the total value of the **discarded bits**
- But since these bits do not terminate, it is impossible to compute the **exact error**.
- However, we can compute the value of the **first bit** of the discarded bits: 0101101...0**0**00**1**01010...

as the **minimum value** of the **error** introduced by the **rounding process** $\Rightarrow 2^{-26} \Rightarrow \sim 1.49 \times 10^{-8}$ (based on my calculator)





3. Special values

➤ Condition: **exp** = 11111111 (single precision)

➤ **Case 1: frac** = 000...0

➤ Represents value $\pm\infty$ (infinity)

➤ Indicates when operation overflows

➤ e.g.: $1.0/0.0 = +\infty$

$-1.0/-0.0 = +\infty$

$1.0/-0.0 = -\infty$

$-1.0/0.0 = -\infty$

➤ **Case 2: frac** $\neq$ 000...0

➤ Not-a-Number (NaN)

➤ Represents cases when no numeric value can be determined

➤ e.g.: $\text{sqrt}(-1)$, $\infty - \infty$, $\infty \times 0$

➤ NaN propagates other NaN

➤ e.g.: $\text{NaN} + x = \text{NaN}$

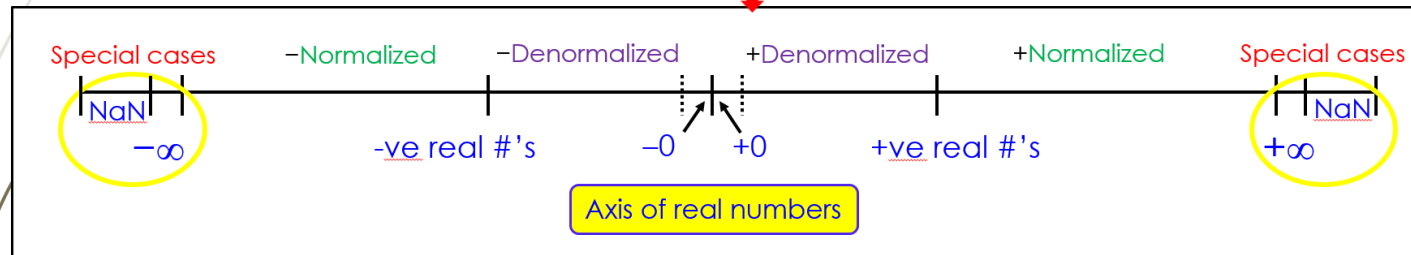
In Lecture 5, we saw that ...

encoding Representing real numbers in memory

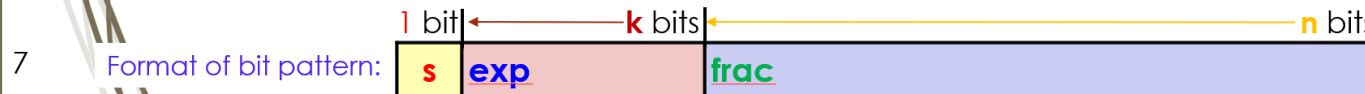
B2R(x) and R2B(x)

- Bit pattern $\Leftrightarrow$ Floating point (IEEE 754) $\Leftrightarrow$ Real numbers
or bit vector
- Encoding scheme called IEEE 754 standard for floating point representation
- Allows us to represent a **large range** of real numbers (but not all):

And most of these
real numbers are
approximated.



- And all represented real numbers (Normalized, Denormalized or Special cases), once converted to a bit pattern, will have this format in memory:



- But not all **R** can be represented!

Which **R** can be represented and which **R** cannot?

- To answer this question, let's examine a simpler IEEE-like encoding scheme:
 - Instead of **w** = 32 (or 64), let's use **w** = 8 and define this encoding scheme as follows:
 - **s** -> 1 bit
 - **exp** -> 4 bits
 - **frac** -> 3 bits
- This simpler IEEE-like encoding scheme would allow us to enumerate all the **R** it can represent and make some observations:
 - smallest/largest denormalized
 - smallest/largest normalized
 - etc.

Which **R** can be represented when **w** = 8

	s	exp	frac	E	Value of R	
Denormalized numbers	0	0000	000	-6	0	smallest denorm closest to zero
	0	0000	001	-6	$1/8 * 1/64 = 1/512$	
	0	0000	010	-6	$2/8 * 1/64 = 2/512$	
	...					
	0	0000	110	-6	$6/8 * 1/64 = 6/512$	largest denorm smallest norm
	0	0000	111	-6	$7/8 * 1/64 = 7/512$	
Normalized numbers	0	0001	000	-6	$8/8 * 1/64 = 8/512$	
	0	0001	001	-6	$9/8 * 1/64 = 9/512$	closest to 1 below
	...					
	0	0110	110	-1	$14/8 * 1/2 = 14/16$	
	0	0110	111	-1	$15/8 * 1/2 = 15/16$	closest to 1 above
	0	0111	000	0	$8/8 * 1 = 1$	
	0	0111	001	0	$9/8 * 1 = 9/8$	
	0	0111	010	0	$10/8 * 1 = 10/8$	largest norm
	...					
	0	1110	110	7	$14/8 * 128 = 224$	
	0	1110	111	7	$15/8 * 128 = 240$	
	0	1111	000	n/a	inf	

Why interpreting **exp** as an unsigned?

$$V = (-1)^s M 2^E$$

Denormalized:

$$E = 1 - \text{bias} = -6$$

$$\text{bias} = 2^{k-1} - 1 = 2^{4-1} - 1 = 7$$

$$M = \text{frac}$$

Normalized:

$$E = \text{exp} - \text{bias}$$

$$\text{bias} = 2^{k-1} - 1$$

$$M = 1. + \text{frac}$$

Conversion in C

- Casting between **int**, **float**, and **double** changes bit pattern
 - **double/float** → **int**
 - Truncates fractional part
 - **int** → **float**
 - Exact conversion, as long as **frac** (obtained when the **int** is normalized) fits in 23 bits
 - Will round according to **rounding rules**
 - **int** → **double**
 - Exact conversion, as long as **frac** (obtained when the **int** is normalized) fits in 52 bits
 - Will round according to **rounding rules**

Demo - C code

- **Conversion** – Observe the change in bit pattern

- `int` → `float`

- `float` → `int`

- **Addition**

- How to?

- <https://www.sciencedirect.com/topics/computer-science/floating-point-addition>

- <https://numeral-systems.com/ieee-754-add/>

- **Associativity** – For floating point numbers $f1$, $f2$ and $f3$:

- Is it always true that $(f1 + f2) + f3 = f1 + (f2 + f3)$?

- Is it always true that $(f1 * f2) * f3 = f1 * (f2 * f3)$?

- **Rounding** – Effect of errors caused by rounding

Floating point arithmetic

- $\mathbf{x} +_{\mathbf{f}} \mathbf{y} = \mathbf{Round}(\mathbf{x} + \mathbf{y})$
- $\mathbf{x} \times_{\mathbf{f}} \mathbf{y} = \mathbf{Round}(\mathbf{x} \times \mathbf{y})$
- Basic idea:
 - First **compute true result**
 - Make it fit into desired precision
 - Possibly overflow if exponent too large
 - Possibly **round to fit into frac**

Summary

- Most real numbers **R** cannot be **exactly** encoded using **IEEE 754 standard floating point representation**, they are **approximated** when encoded

- This happens because most **R** need more than 23 bits to encode the binary version of their fractional part (**frac**)

- Therefore their **frac** is subject to **rounding**, hence **approximation**

1. **Round up**: add 1 to bit at **rounding position** and discard bits on right of **rounding position**
2. **Round down**: discard bits on right of **rounding position**
3. **Round to even number**: either **Round up** or **Round down**

- **Denormalized** values = $\pm 0.\text{frac} \times 2^{-126}$ (single precision)

- Condition: **exp** = 00000000

- **Case 1**: **frac** = 000...0 $\rightarrow$ +0 and -0

- **Case 2**: **frac** $\neq$ 000...0 $\rightarrow$ #s closest to 0.0

- All numbers equidistant because all have same 2^E and their **M** increments linearly

- **Special values** (single precision)

- Condition: **exp** = 11111111

- **Case 1**: **frac** = 000...0 $\rightarrow$ $+\infty$ and $-\infty$ (infinity)

- **Case 2**: **frac** $\neq$ 000...0 $\rightarrow$ NaN (Not-a-Number)

Appropriate Equations:

$$E = 1 - \text{bias}$$

$$\text{bias} = 2^{k-1} - 1$$

$$M = \text{frac}$$

➤ Smallest Denormalized value: 0 00000000 000000000000000000000001

$$V = (-1)^s M 2^E = 0.000000000000000000000001 \times 2^{-126} = \sim 1.4 \times 10^{-45}$$

➤ Largest Denormalized value: 0 00000000 111111111111111111111111

$$V = (-1)^s M 2^E = 0.111111111111111111111111 \times 2^{-126} = \sim 1.18 \times 10^{-38}$$

Next Lecture

➤ Introduction

- C program -> assembly code -> machine level code

➤ Assembly language basics: data, `move` operation

- Memory addressing modes

➤ Operation `leaq` and Arithmetic & logical operations

➤ Conditional Statement – Condition Code + `cmovX`

➤ Loops

➤ Function call – Stack

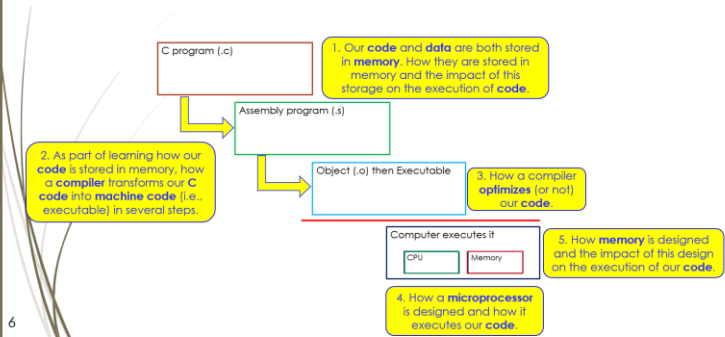
- Overview of Function Call
- Memory Layout and Stack - x86-64 instructions and registers
- Passing control
- Passing data – Calling Conventions
- Managing local data
- Recursion

➤ Array

➤ Buffer Overflow

➤ Floating-point data & operations

In CMPT 295, we shall learn ...



6